Risk Management Plan:

Email Approval Automation System

Overview
This document identifies potential risks in the system and outlines strategies to mitigate them. The goal is to ensure system reliability, security, and performance throughout development and deployment.

1. Technical Risks

Risk: Integration issues between frontend, backend, and n8n workflow
Impact: System components may not communicate correctly
Mitigation:

- Clearly define API contracts
- Test endpoints independently before integration
- Use mock data during early development

Risk: AI service failure or unreliable outputs
Impact: Incorrect email classification or approval decisions
Mitigation:

- Validate AI outputs before storing results
- Use fallback logic (rule-based decisions)
- Allow manual override of decisions (Phase 2)

Risk: Data mismatch between n8n workflow and Supabase schema
Impact: Workflow outputs may not match database fields, causing insertion errors
Mitigation:

- Align n8n output format directly with Supabase schema (EmailRequest, AIAnalysis)
- Use validation layer in backend before inserting data
- Log failed insertions for debugging

2. Performance Risks

Risk: System cannot handle high user traffic
Impact: Slow response times or system crashes
Mitigation:

- Use horizontal scaling for backend services
- Implement load balancing
- Use asynchronous processing for workflows

Risk: Slow database queries
Impact: Delays in fetching or storing data
Mitigation:

- Use indexed queries
- Optimize database schema
- Implement caching where needed

3. Security Risks

Risk: Exposure of API keys (AI or database)
Impact: Unauthorized access or misuse of services
Mitigation:

- Store all API keys in environment variables
- Never expose keys in frontend code
- Restrict access through backend only

Risk: Unauthorized access to user data
Impact: Data privacy issues
Mitigation:

- Implement authentication and authorization
- Use secure tokens (JWT)
- Protect all sensitive routes

4. Workflow Risks

Risk: n8n workflow failure or misconfiguration
Impact: Email requests not processed correctly
Mitigation:

- Add logging and monitoring for workflows
- Test workflows with multiple scenarios
- Implement retry mechanisms

Risk: Incorrect or inconsistent data format between n8n, backend, and Supabase

Impact: Workflow runs successfully but data is stored incorrectly or fails to save to the database
Mitigation:

- Define strict JSON structure for all workflow outputs
- Validate data before inserting into Supabase
- Test payload formats between n8n and backend endpoints
- Use logging to detect malformed or missing fields

5. Team / Project Risks

Risk: Lack of coordination between team members
Impact: Delays or duplicated work
Mitigation:

- Use GitHub and Jira for task tracking
- Maintain regular team communication
- Clearly assign responsibilities

Conclusion
By identifying risks early and implementing mitigation strategies, the system can maintain reliability, security, and performance throughout development.